DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Models of Parallel Processing

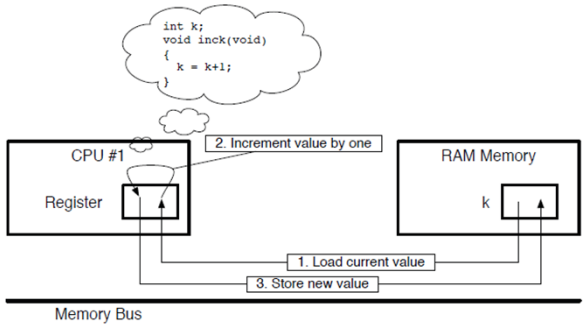
- Race conditions
- Mutual exclusion
- HW mutual exclusion implementation
- SW mutual exclusion implementation
- Semaphores
- Monitors and Condition Synchronization
- Message passing communication

Task Interaction

- In both general-purpose and real-time systems, processes do not live by themselves and are not independent of each other.
- Rather, several processes are brought together to form the application software, and they must therefore cooperate to solve the problem at hand.
- Processes must therefore be able to communicate, that is, exchange information in a meaningful way.
- We have already seen that it is possible to share some memory among processes in a controlled way by making part of their address space refer to the same physical memory region, if separate processes, or share global variables if threads within the same process
- However, this is only part of the story. In order to implement a correct and meaningful data exchange, processes must also synchronize their actions in some ways.
- For instance, they must not try to use a certain data item if it has not been set up properly. Otherwise inconsistent data may be accessed
  - e.g a linked list in shared memory requires proper link setting that may be corrupted when the two tasks try to concatenate a new element concurrently
- Errors due to the unexpected interleaving of actions over shared data structures are often referred to as ***race conditions***

A race condition example

- Suppose a very simple function void **`inck(void)`** that only contains the statement **`k = k+1`**.
- No real-world CPU is actually able to increment k in a single, indivisible step, at least when the code is compiled into ordinary assembly instructions.
- A simplified computer based on the von Neumann architecture will perform a sequence of three distinct steps:
  - 1. Load the value of k from memory into an internal processor register; From the processor's point of view, this is an external operation because it involves both the processor itself and memory The load operation is not destructive, that is, k retains its current value after it has been performed.
  - 2. Increment the value loaded from memory by one. Unlike the previous one, this operation is internal to the processor.
  - 3. Store the new value of k into memory with an external operation involving a memory bus transaction like the first one. It is important to notice that the new value of k can be observed from outside the processor only at this point, not before. In other words, if we look at memory, k retains its original value until this final step has been completed.



Race condition example - cont.

The following sequence of events may occur:

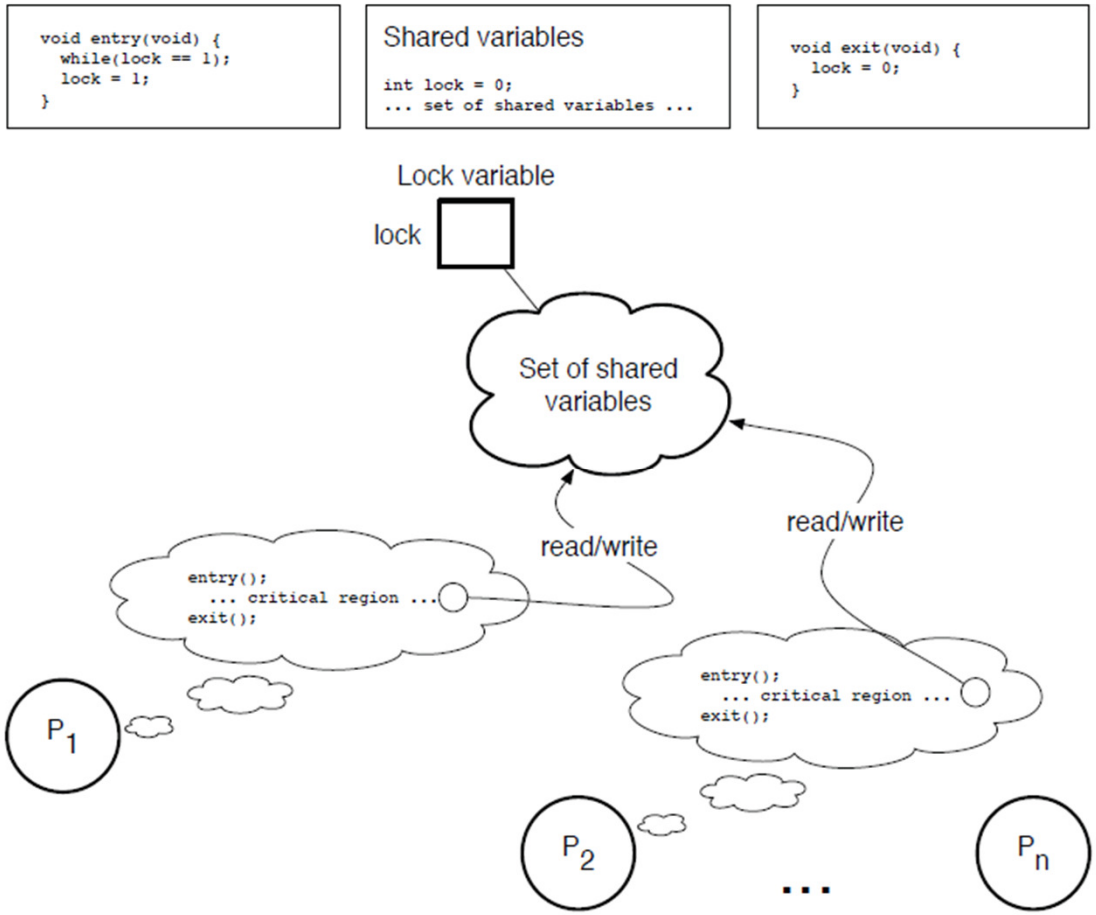
1. CPU #1 loads the value of k from memory and stores it into one of its registers, R1. Since k currently contains 0, R1 will also contain 0.
2. CPU #1 increments its register R1. The new value of R1 is therefore 1.
3. Now CPU #2 takes over, loads the value of k from memory, and stores it into one of its registers, R2. Since CPU #1 has not stored the updated value of R1 back to memory yet, CPU #2 still gets the value 0 from k, and R2 will also contain 0.
4. CPU #2 increments its register R2. The new value of R2 is therefore 1.
5. CPU #2 stores the contents of R2 back to memory in order to update k, that is, it stores 1 into k.
6. CPU #1 does the same: it stores the contents of R1 back to memory, that is, it stores 1 into k.

- Looking closely at the sequence of events just described, it is easy to notice that taking an (obviously correct) piece of sequential code and using it for concurrent programming in a careless way did not work as expected.
    - In the particular example just made, the final value of k is clearly incorrect: the initial value of k was 0, two distinct processes incremented it by one, but its final value is 1 instead of 2, as it should have been.
      Even worse: the result of the concurrent update is not only incorrect but it is incorrect only sometimes.
- Finally, observe that the same problem may arise with two processes/treads on the same processor

The need of mutual exclusion

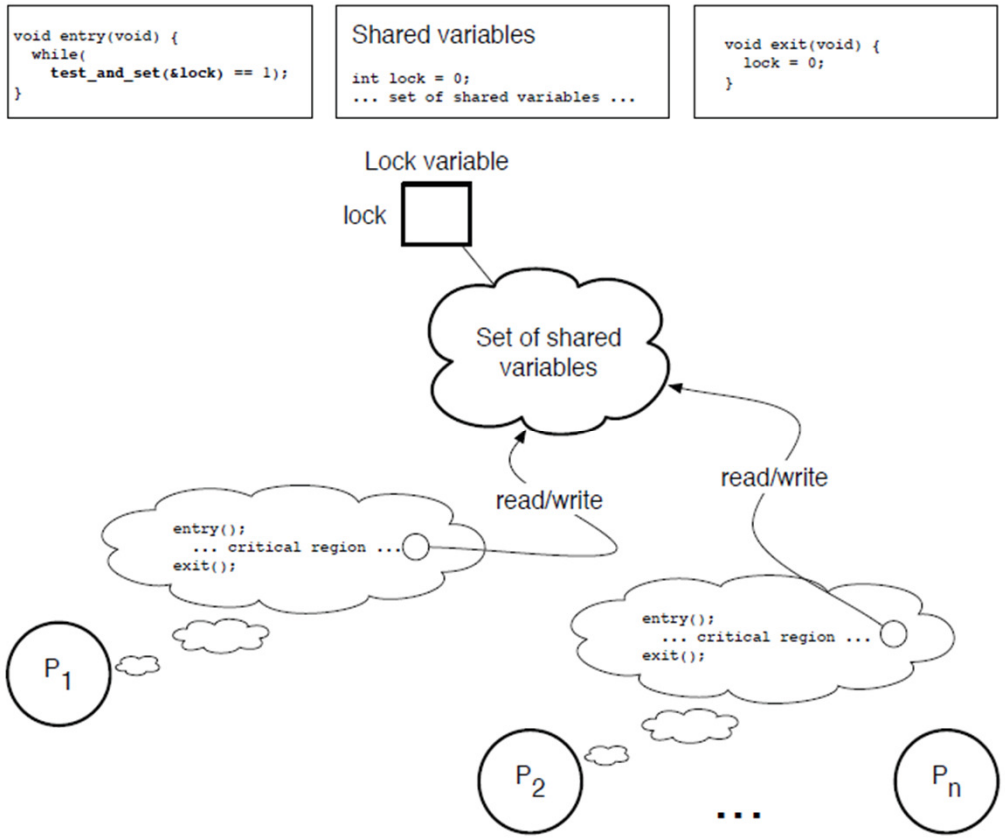
- A general solution is to protect **critical sections**, i.e. sections of code accessing shared information by means of **locks**, thus ensuring that only a task at a time is allowed to execute a section of code 'protected' by that lock
- A task that wants to access a shared object, by means of a certain critical region, must perform the following sequence of steps:
  - Acquire some sort of lock associated with the shared object and wait if it is not immediately available.
  - Use the shared object.
  - Release the lock, so that other processes can acquire it and be able to access the same object in the future
- In any case, four conditions must be satisfied in order to have an acceptable solution:
  - 1. It must really work, that is, it must prevent any two processes from simultaneously executing code within critical regions pertaining to he same shared object.
  - 2. Any process that is busy doing internal operations, that is, is not currently executing within a critical region, must not prevent other processes from entering their critical regions, if they so decide.
  - 3. If a process wants to enter a critical region, it must not have to wait forever to do so. This condition guarantees that the process will eventually make progress in its execution.
  - 4. It must work regardless of any low-level details about the hardware or software architecture. For example, the correctness of the solution must not depend on the number of processes in the system, the number of physical processors, or their relative speed.

A Naive Implementation



Hardware assisted solution

- Several processor provides a **`test_and_set`** instruction.
- This instruction has the address p of a memory word as argument and atomically performs the following three steps:
  - 1. It reads the value v of the memory word pointed by p.
  - 2. It stores 1 into the memory word pointed by p.
  - 3. It puts v into a register.
- An alternative solution uses XCHG instruction, which atomically exchanges the contents of a register with the contents of a memory word
- Both Instruction ensure atomicity even in a multicore architecture because they internally lock the memory bus



Software Solution: the Peterson algorithm

```
//Global Variables
int flag[2] = {0, 0};
int turn=0;

//Enter critical section
void entry(int pid) {
    int other = 1-pid;
    flag[pid] = 1;
    turn = pid;
    while(flag[other] == 1
      && turn == pid);
}

//Exit critical section
void exit(int pid) {
    flag[pid] = 0;
}
```

Lemma: ***pid in critical section => flag[pid] == 1***

Proof: pid in critical section => pid in critical section **or** pid in entry after instruction flag[pid] = 1 => flag[pid] == 1 (Invariant)

Theorem: At most one task is in critical section

Proof: At the time the first task just enters in critical section, the other task is either:

1. not calling entry() or in any case before instruction turn = pid
2. calling entry and after instruction turn = pid

In case 1 if the other task enters the while loop it finds the condition false: flag[other] = 1 (lemma) and turn == pid (turn is not changed by the other task in critical section)

In case 2 turn == pid, otherwise the first task would not have entered the critical section. For the same reason as before, the while loop finds the condition false

- Peterson algorithm is shown here for 2 tasks, but a similar algorithm is valid for N tasks
- Unlike previous HW solution, fairness is guaranteed here. A slower task is guaranteed to eventually enter the critical section (because of the variables **`flags`** and **`turn`**)

The Peterson algorithm for more than two processes

```
//Shared Variables
int level[N]
int last_to_enter[N-1]

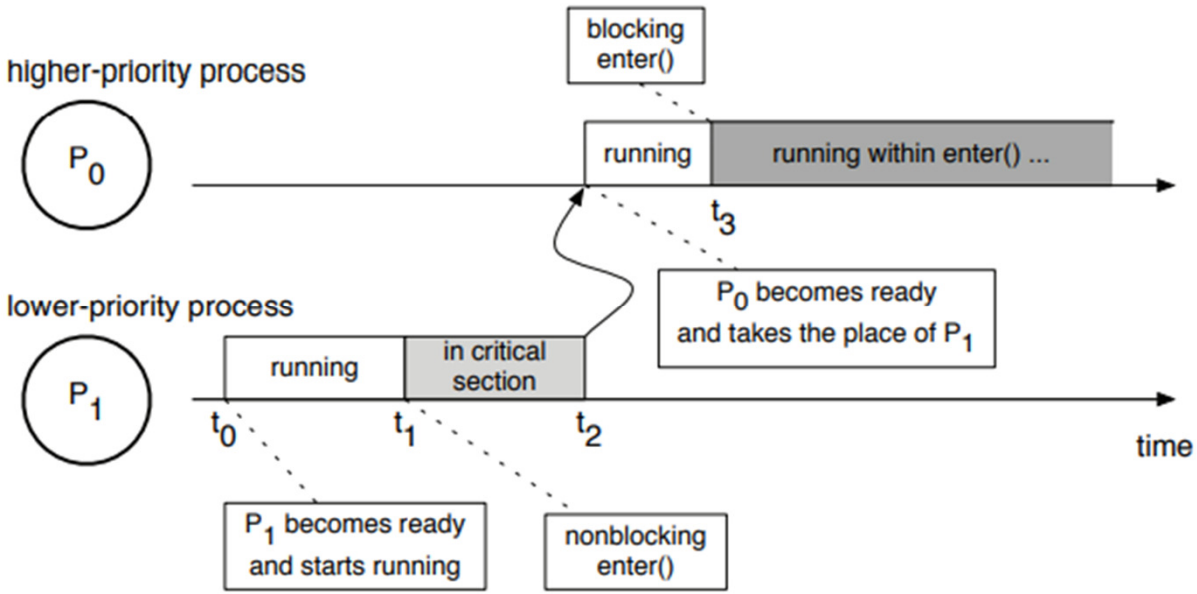
void lock(pid)  //pid from 0 to N-1
{
          int h;
          for(h = 0; h < N-1; h++)
          {
                    level[pid] = h;
                    last_to_enter[h] = pid;
                    while(last_to_enter[h] == pid &&  there exists k ≠ pid, such that
level[k] ≥ h);
          }
}

void unlock(int pid) {level[pid]= -1;}
```

From Active to Passive Wait

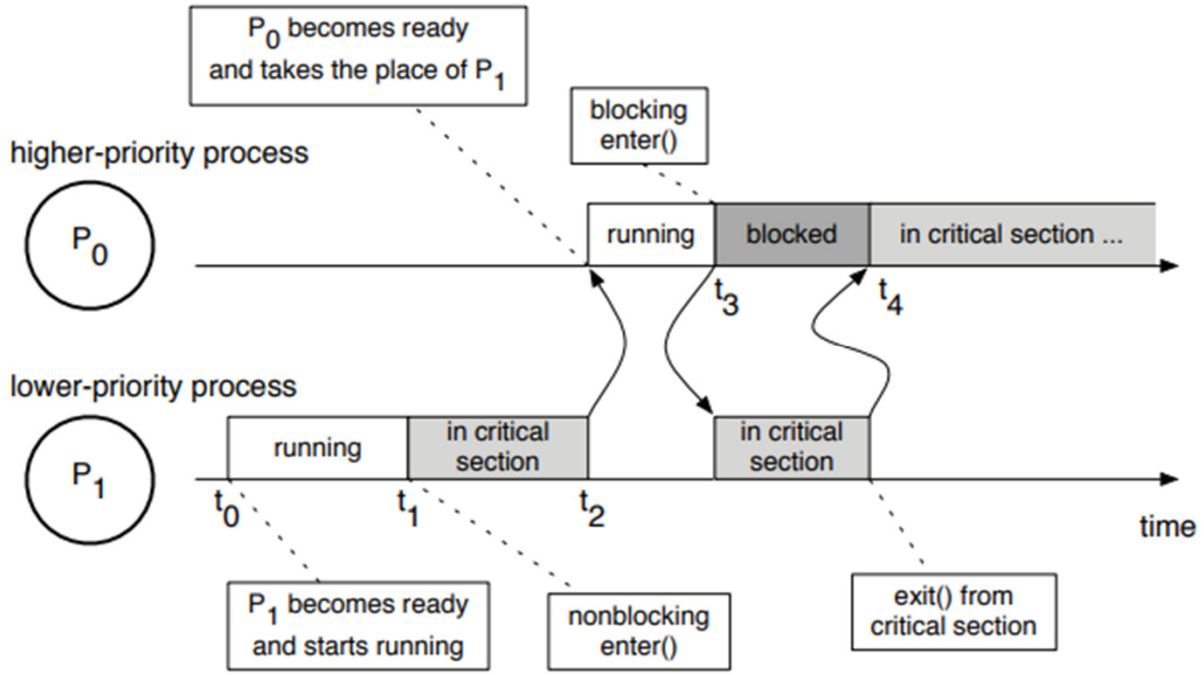
- The solutions seen so far imply a repeated check of s shared variable.
- This mechanism is similar to the polling I/O synchronization, but may waste processor cycles
  - Moreover, when the tasks share the same processor, the wait loop may affect the execution of the critical section leading to unbounded access time to the critical section
- A better solution would imply the OS removing the processor from a task trying to access a critical section already locked until it becomes free
- This implies making a request to the OS for entering and exiting a critical section
- In a single processor system OS data structures will replace the shared variables, and any task (process or thread) willing to access a critical section will be put in wait state if another task already gained access. When a task exits the critical section, the OS shall be informed, and tasks waiting for that critical section will be made ready again and eventually they will try again to enter the critical section.
- The implementation of the OS code for multicore systems is more complicated because the OS code itself is distributed
  - In this case HW specific locking instructions such as atomic exchanges shall be used internally in the OS as well as polling (***spinlock***)

Busy Wait for two processes with fixed priority



Indefinite wait (deadlock) may happen if the two processes share the same processor

Passive Wait for two processes with fixed priority



In this case dedlock does not occur

Semaphores

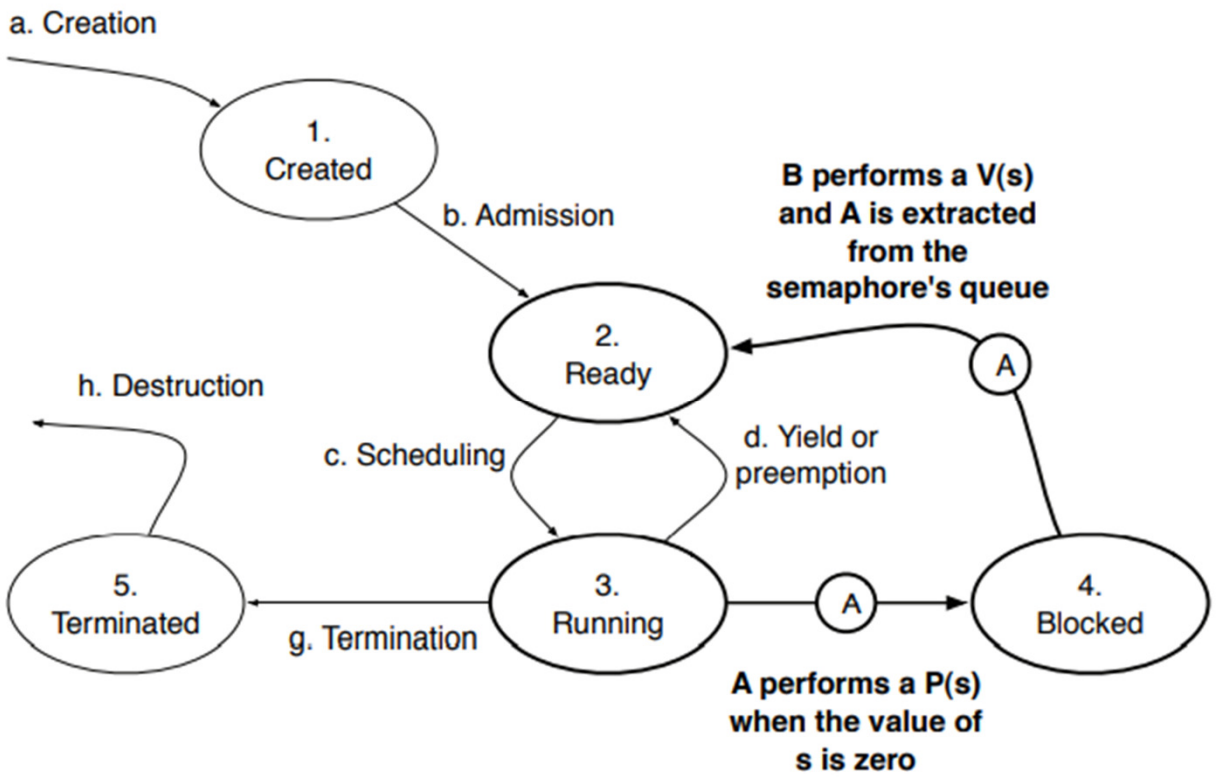
- A semaphore is an object that comprises two abstract items of information:
  - 1. a nonnegative, integer value;
  - 2. a queue of processes passively waiting on the semaphore.

- Upon initialization, a semaphore acquires an initial value specified by the programmer, and its queue is initially empty.

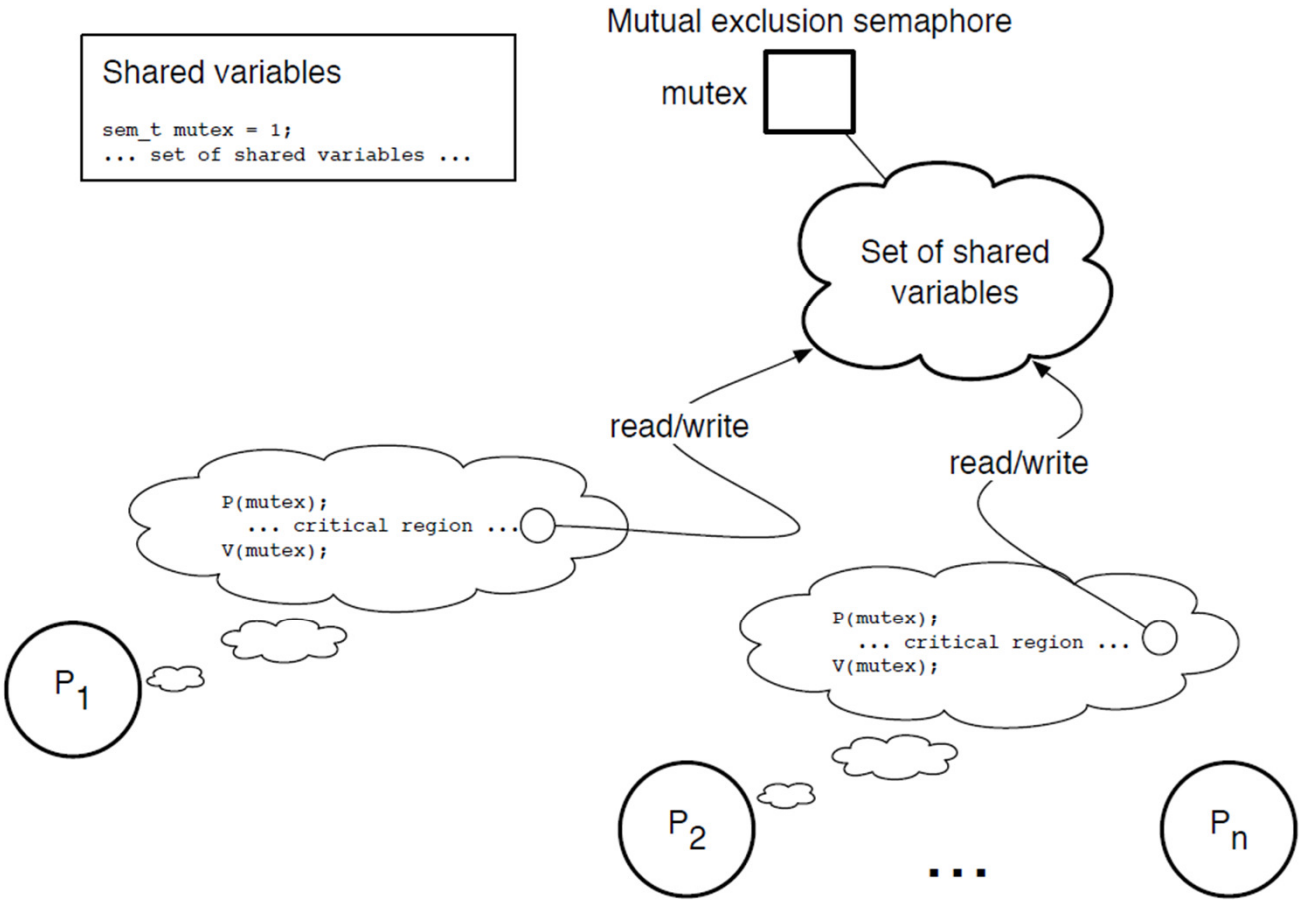
- Neither the value nor the queue associated with a semaphore can be read or written directly after initialization.

- On the contrary, the only way to interact with a semaphore is through the following two primitives that are assumed to be executed atomically:
  - 1. P(s), when invoked on semaphore s, checks whether the value of the semaphore is (strictly) greater than zero. If this is the case, it decrements the value by one and returns to the caller without blocking. Otherwise, it puts the calling process into the queue associated with the semaphore and blocks it by moving it into the Wait (Blocked) state of the process state diagram.
  - 2. V(s), when invoked on semaphore s, checks whether the queue associated with that semaphore is empty or not. If the queue is empty, it increments the value of the semaphore by one. Otherwise, it picks one of the blocked processes found in the queue and makes it ready for execution again by moving it into the Ready state of the process state diagram.

Semaphores and Process states



Using Semaphores to protect a critical section



Using Semaphores for condition synchronization

- Besides mutual exclusion, semaphores are also useful for condition synchronization, that is, when we want to block a process until a certain event occurs or a certain condition is fulfilled.
- For example, considering the producers–consumers problem, it may be desirable to block any consumer that wants to get a data item from the shared buffer when the buffer is completely empty, instead of raising an error indication.
    - A blocked consumer must be unblocked as soon as a producer puts a new data item into the buffer.
- Symmetrically, we might also want to block a consumer when it tries to put more data into a buffer that is already completely full.
- In order to do this, we need one semaphore for each synchronization condition that the concurrent program must respect. In this case, we have two conditions, and hence we need two semaphores:
    - 1. The semaphore empty counts how many empty elements there are in the buffer. Its initial value is N because the buffer is completely empty at the beginning. Producers perform a P(empty) before putting more data into the buffer to update the count and possibly block themselves, if there is no empty space in the buffer. After removing one data item from the buffer, consumers perform a V(empty) to either unblock one waiting producer or increment the count of empty elements.
    - 2. Symmetrically, the semaphore full counts how many full elements there are in the buffer. Its initial value is 0 because there is no data in the buffer at the beginning. Consumers perform P(full) before removing a data item from the buffer, and producers perform a V(full) after storing an additional data item into the buffer.

The producer - consumer example

```
void prod(int d) {
    P(empty);
    P(mutex);

    buf[in] = d;
    in = (in+1) % N;

    V(mutex);
    V(full);
}
```

```
#define N 8
int buf[N];
int in=0, out=0;
sem_t mutex=1;
sem_t empty=N;
sem_t full=0;
```

```
int cons(void) {
    int c;

    P(full);
    P(mutex);

    c = buf[out];
    out = (out+1) % N;

    V(mutex);
    V(empty);

    return c;
}
```

Semaphores in Linux

- **`sem_t *sem_open(const char *`*`name`*`, int `*`oflag`*`, mode_t `*`mode`*`, unsigned int `*`value`*`)`**
  Open a named semaphore

- **`int sem_close(sem_t *`*`sem`*`);`**
  Close a unnamed semaphore

- **`int sem_init(sem_t *`*`sem`*`, int `*`pshared`*`, unsigned int `*`value`*`);`**
  Initialize an unnamed semaphore

- **`int sem_post(sem_t *`*`sem`*`);`**
  V Operation

- **`int sem_wait(sem_t *`*`sem`*`);`**
- **`int sem_trywait(sem_t *`*`sem`*`);`**
- **`int sem_timedwait(sem_t *restrict `*`sem`*`, const struct timespec *restrict `*`abs_timeout`*`);`**
  P Operation

Monitors

- Semaphore represent a rather ‘low level’ approach for synchronization and lead to complicated solutions with increased risk of bugs.

- To address these issues, a higher-level and more structured interprocess communication mechanism, called monitor, was proposed by Brinch Hansen and Hoare.
  - It is interesting to note that, even if these proposals date back to the early ’70s, they were already based on concepts that are common nowadays and known as object-oriented programming.

- In its most basic form,a monitor is a composite object and contains
  - a set of shared data;
  - a set of methods that operate on them.

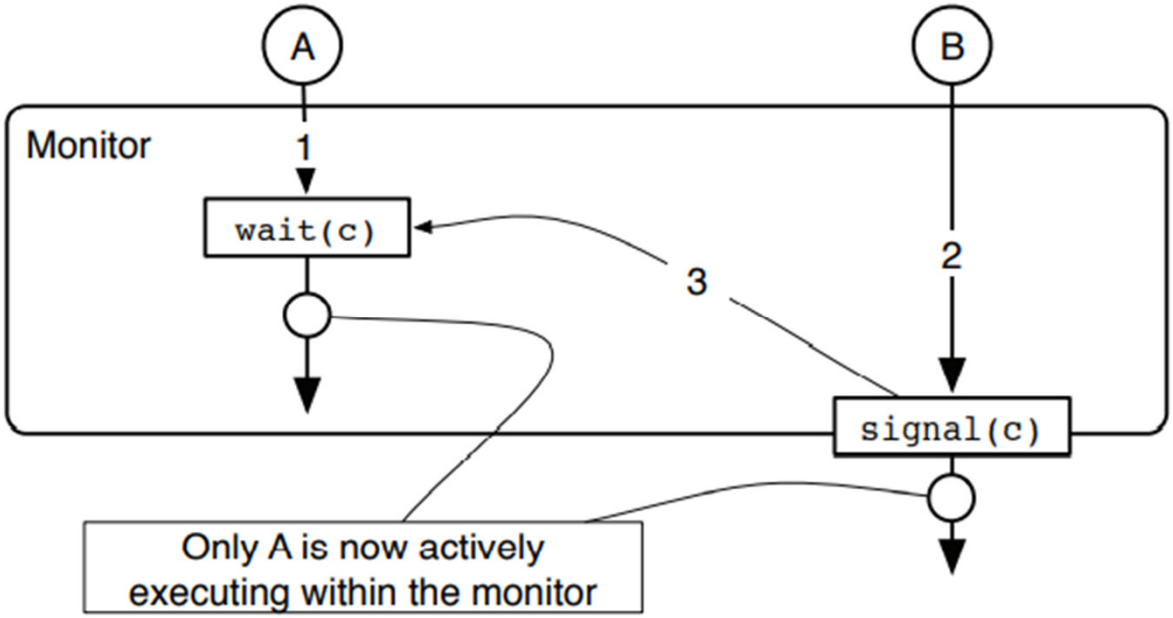
- With respect to its components, a monitor guarantees the following two main properties:
  - Information hiding, because the set of shared data defined in the monitor is accessible only through the monitor methods and cannot be manipulated directly from the outside. Monitor methods are not hidden and can be freely invoked from outside the monitor.
  - Mutual exclusion among monitor methods, that is, the monitor implementation, must guarantee that only one process will be actively executing within any monitor method at any given instant.

- No direct support in C++, but the java synchronized method closely reflect the monitor approach

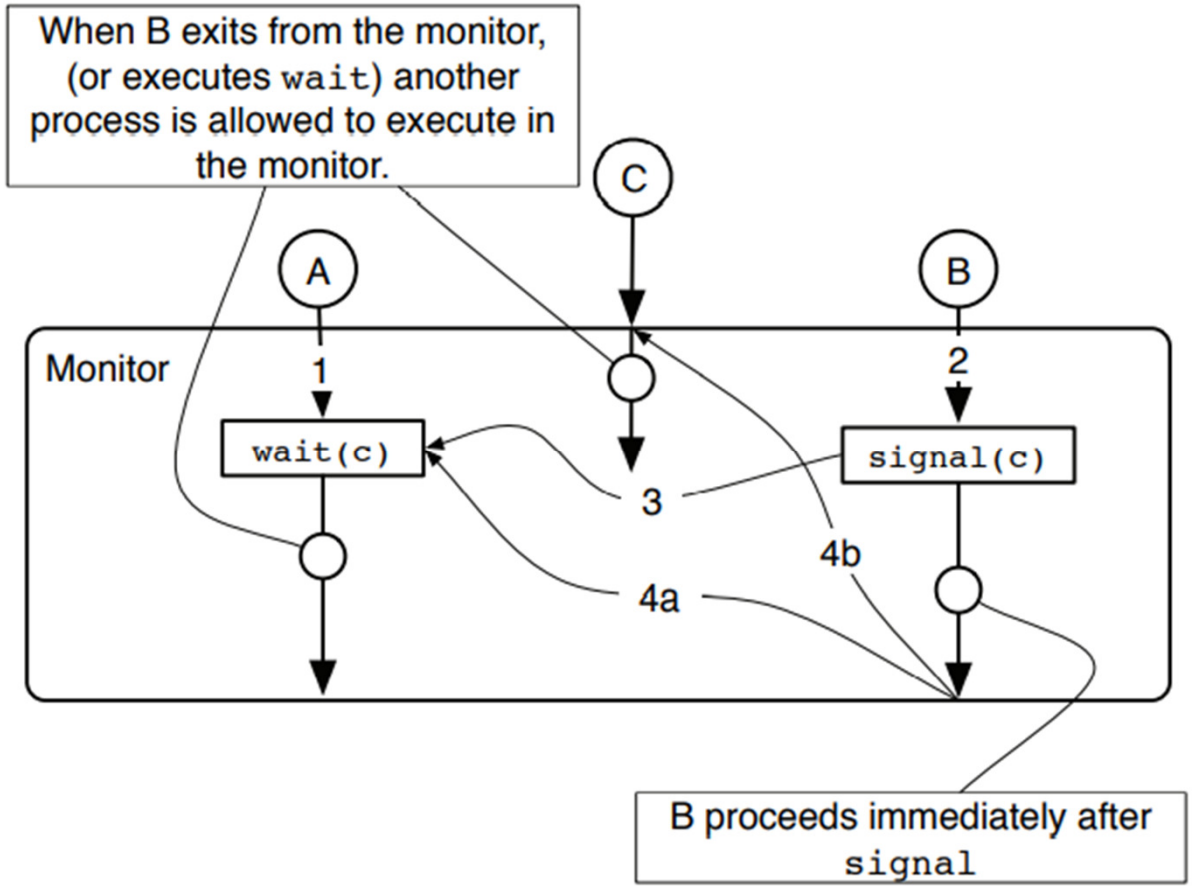
Synchronization in monitors

- We have seen that semaphores can be also used for synchronization
- Synchronization is achieved also in monitors by means of the ***condition variables***.
- Condition variables can be used only by the methods of the monitor they belong to, and cannot be referenced in any way from outside the monitor boundary. They are therefore hidden exactly like the monitor's shared data.
- The following two primitives are defined on a condition variable c:
  - **`wait(c)`** blocks the invoking process and releases the monitor in a single, atomic action.
  - **`signal(c)`** wakes up one of the processes blocked on c; it has no effect if no processes are blocked on c.
- The informal reasoning behind the primitives is that, if a process starts executing a monitor method and then discovers that it cannot finish its work immediately, it invokes wait on a certain condition variable. In this way, it blocks and allows other processes to enter the monitor and perform their job. When one of those processes, usually by inspecting the monitor's shared data, detects that the first process can eventually continue, it calls signal on the same condition variable.
- **`wait()`** and **`signal()`** are implemented as java object methods and in this case the condition variable is the object instance itself
- Condition variables are also provided by POSIX for thread synchronization, but the Monitor abstraction is not pushed further, and mutexes will be defined to handle critical sections

Two possible synchronization implementations (1)



Two possible synchronization implementations (2)



Producer-Consumer implementation using a Monitor

```
#define N 8

monitor ProducersConsumers
{
    int buf[N];
    int in = 0, out = 0;
    condition full, empty;
    int count = 0;

    void produce (int v)
    {
        if(count == N) wait( empty );
        buf[in] = v;
        in = (in + 1) % N;
        count = count + 1;
        if(count == 1) signal (full );
    }
```

```
int consume (void)
  {
        int v;

        if(count == 0) wait(full );

        v = buf[out];
        out = (out + 1) % N;
        count = count - 1;

        if(count == N -1) signal (empty );

        return v;
  }

}
```

Observe that the check before the wait() operation in producer and consumer works only if at most one task is awakened by the signal operation. If more than one task is possibly awakened, it is necessary to use the **while** operation, i.e., repeating the test. If more than one task is possibly awakened and an **if** operation were used instead, more than one waiting task may be awakened and incorrectly proceed. (e.g. two waiting consumers just after a producer added a single item to an empty buffer). Using **while** instead of **if** is correct in any case.

Mutexes and Conditions in POSIX

- Creation of a mutex variable (pthread_mutex_t)

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *
attr);
```

- Mutex lock/unlock/trylock

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- Creation of a condition variable

```
int pthread_cond_init(pthread_cond_t *restrict cond, const pthread_condattr_t
*restrict attr);
```

- Condition wait

```
int pthread_cond_wait(pthread_cond_t * cond, pthread_mutex_t *mutex);
```

- Condition signal

```
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
```

Producer Consumer in Java

```
class prodcon{
   int N = 80;
   int buffer[] = new int[N];
   int numItems = 0;
   int in = 0, out = 0;

   synchronized void produce(int item)
   {
      while(numItems == N)
      {
         try {
            wait();
         }catch(Exception exc){}
      }
      buffer[in] = item;
      in = (in+1)%N;
      numItems++;
      notify();
   }
```

```
   synchronized int  consume()  {
      while(numItems == 0)   {
         try {
            wait();
         } catch(Exception exc){}
      }
      int retItem = buffer[out];
      out = (out+1)%N;
      numItems--;
      notify();
      return out;
   }
}
```

```
public static void main(String args[])  {
      prodcon pc = new prodcon();
      Thread producer = new Thread() {
         public void run() {
            for(int i = 0; i < 100; i++)   {
               pc.produce(i);
            }
         }
      };
      Thread consumer = new Thread() {
         public void run() {
            for(int i = 0; i < 100; i++)  {
               int retItem = pc.consume();
            }
         }
      };
      producer.start();
      consumer.start();
      try {
         producer.join();
         consumer.join();
      }catch(Exception exc){}
   }
}
```

Producer Consumer in Java - alternate

```
import java.util.concurrent.locks.*;
class prodcon1 {
    int N = 80;
    int buffer[] = new int[N];
    int numItems = 0;
    int in = 0, out = 0;
    ReentrantLock lock = new ReentrantLock();
    Condition full = lock.newCondition();
    Condition empty = lock.newCondition();
    void produce(int item)  {
        lock.lock();
        while(numItems == N) {
            try {
                empty.await();
            }catch(Exception exc){}
        }
        buffer[in] = item;
        in = (in+1)%N;
        numItems++;
        full.signal();
        lock.unlock();
    }
    int  consume()  {
        lock.lock();
        while(numItems == 0)  {
            try {
                full.await();
            } catch(Exception exc){}
        }
        int retItem = buffer[out];
        out = (out+1)%N;
        numItems--;
        empty.signal();
        lock.unlock();
        return out;
    }
}
```

Producer Consumer in Python

```
import threading

class ProdCon():
  def __init__(self):
    self.buffer = []
    self.numItems = 0
    self.lock = threading.RLock()
    self.full = threading.Condition(lock =
self.lock)
    self.empty = threading.Condition(lock =
self.lock)
    self.N = 20

  def produce(self, item):
    self.lock.acquire()
    while len(self.buffer) == self.N:
        self.empty.wait()
    self.buffer.append(item)
    self.full.notify()
    self.lock.release()
```

```
  def consume(self):
    self.lock.acquire()
    while len(self.buffer) == 0:
        self.full.wait()
    retItem = self.buffer.pop()
    self.empty.notify()
    self.lock.release()
    return retItem

class Producer(threading.Thread):
  def __init__(self, prodcon):
   threading.Thread.__init__(self)
   self.prodcon = prodcon

  def run(self):
    for i in range(100):
      self.prodcon.produce(i)
```

```
class Consumer(threading.Thread):
  def __init__(self, prodcon):
    threading.Thread.__init__(self)
    self.prodcon = prodcon

  def run(self):
    for i in range(100):
      retItem =
self.prodcon.consume()


prodcon = ProdCon()
producer = Producer(prodcon)
consumer = Consumer(prodcon)
consumer.start()
producer.start()
producer.join()
consumer.join()
```

Message Passing Synchronization

- It represent an alternative way to shared memory solutions.
  - A mandatory solution when actors reside on different machines not sharing memory

- In its simplest, and most abstract, form a message-passing mechanism involves two basic primitives:
  - a **send** primitive, which sends a certain amount of information, called a message, to another process;
  - a **receive** primitive, which allows a process to block waiting for a message to be sent by another process, and then retrieve its contents.

- Even if this definition still lacks many important details it is already clear that the most apparent effect of message passing primitives is to transfer a certain amount of information from the sending process to the receiving one.

- At the same time, the arrival of a message to a process also represents a synchronization signal because it allows the process to proceed after a blocking receive.

- The last important requirement of a satisfactory interprocess communication mechanism, mutual exclusion, is not a concern here because messages are never shared among processes, and their ownership is passed from the sender to the receiver when the message is transferred. In other words, the mechanism works as if the message were instantaneously copied from the sender to the receiver
  - even if real-world message passing systems do their best to avoid actually copying a message for performance reasons

Main design choices in message passing

1. For a sender, how to identify the intended recipient of a message. Symmetrically, for a receiver, how to specify from which other processes it is interested in receiving messages. In more abstract terms, how a process naming scheme must be defined.
    a. how the send and receive primitives are associated to each other;
    b. their symmetry (or asymmetry).

2. The synchronization model, that is, under what circumstances communicating processes shall be blocked, and for how long, when they are engaged in message passing.

3. How making a message content consistent (endianity and pointers)

Direct vs indirect naming

- The most straightforward approach is for the sending process to name the receiver directly
  - For example, in a IP communication the sender shall specify the Internet Protocol (IP) address and port of the intended receiver.

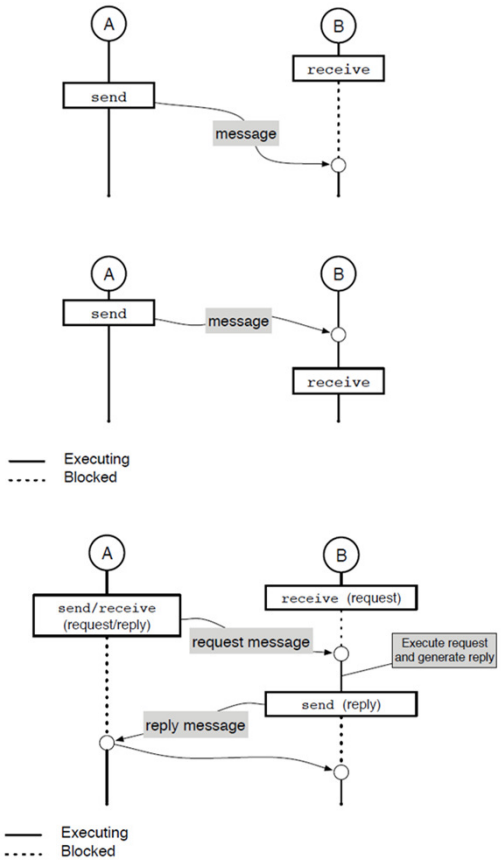
- Considering the receiver, direct naming is not so straightforward
  - Normally the receiver provided a service that may be accessed by different clients
  - After a connection has been established sender and receiver exchange messages and can therefore swap their roles
  - Both must adhere to a **communication protocol** in order to achieve a consistent communication

- Indirect naming means that the sender does not address the receiver explicitly.
  - For example in a multicast communication, a set of receivers may receive the messages sent by the server, that may also be unaware of the identity of the recipients

- **Publish-Subscribe** is a common pattern in this context
  - A Publisher offers some kind of service
  - A Subscriber can subscribe for a given service and shall receive messages issued by the publisher
  - More than one subscriber may be registered and thew publisher may be unaware of this, especially when association between publishers and subscribers is implemented by a separate **broker**.

Message Synchronization

- The receiver normally needs to synchronize to the message receipt
  - Asynchronous receive shall define a callback to be called asynchronously when a message has been received
- Normally send() call returns soon, unless internal message queuing reaches a given limit
- In randezvous schema, the sender synchronizes with the message reception reception
- Normally randezvous schema are implemented with a acknowledge message sent back by the receiver
- The communication protocol specifies the messages to be sent and received in order to achieve the required synchronization



Message Buffering

- Very often the send() routine will return before the message has been received
- If communication occurs within the same computer, this means temporarily storing exchanged packets in memory (managed by the OS)
- If communication involves different computers, packets may be stored in any point of the path between the sender and the receiver
- Considering network communication, messages are often buffered in routers before being dispatched at the right port, based on the message header and the routing information (store and forward)
- Other routers start dispatching the message as soon as the header has been received and the destination can be computed (cut-through)
- Message buffering is normally transparent to the sender and the receiver and therefore no assumption can be done about message transfer time.
- If messages are used for synchronization, the use of Acknlowledge return messages is required

Pros and Cons of Message Buffering

- Having a large buffer between the sender and the receiver decouples the two processes and, on average, makes them less sensitive to any variation in execution and message passing speed.
- Therefore it increases the likelihood of executing them concurrently without unnecessarily waiting for one another.
- The interposition of a buffer increases the message transfer delay and makes it less predictable.
- For some synchronization models, the amount of buffer space required at any given time to fulfill the model may depend on the processes' behavior and be very difficult to predict.
- For the purely asynchronous model, the maximum amount of buffer space to be provided by the system may even be unbounded in some extreme cases. This happens, for instance, when the sender is faster than the receiver so that it systematically produces more messages than the receiver is able to consume

Reliable vs Unreliable message transfer

- The transmission of a message is subject to several possible communication errors
- Effect of communication errors can be:
  - Message corruption
  - Messages lost
  - messages duplicated

- Reasons for for communication errors can be:
  - transmission errors
  - message discarded in internal router buffers when the traffic is too high

- Reliability can be granted on not depending on the communication protocol adopted
  - TCP/IP guarantees reliable delivery and non corrupted messages
  - UDP guarantees only that received messages are not corrupted (checksum handled in the protocol) but no guarantee is given about message delivery, i.e. messages can be lost. For Local Area Networks, the reason for the loss of a message is exclusively internal router buffer overrun

Reliability vs real-time communication

- Real-time communication requires that a message shall be successfully received within a given amount of time.
- Reliability in communication requires detecting whether messages are lost...
  - Normally assigning an increasing timestamp value to each message so that missed messages can be detected checking timestamp
- ...and handling the message loss
  - Normally by requesting the missed message
- However this added reliability layer introduce a unbounded delay even if the transmission time is bounded, therefore breaking real-time requirements
- On the other side, loss of messages cannot often be avoided for sure
- This has to be taken into account in communication for real-time distributed systems
- Sometimes a (limited) loss of messages can be tolerated, e.g. when then system is performing closed loop control
  - Missing messages can be considered an added noise in sensor readout or actuator command and may be tolerated provided the overall controllow introduces a stability margin large enough
- Moreover, for LAN communication, by insulating the system from external (unpredictable) traffic it is in general possible to make sure that not internal buffer overrun occurs in the involved routers.
- For this reason, UDP is often adopted as communication protocol in real-time systems.

Message content consistency

- Not always the bytes forming a message bring consistent information, especially when the receiver resides on a different machine

- A common error is due to the endianity that may be different between the machines involved in communication.
  - char arrays always works, but multybyte numbers can be interpreted differently (big/little endian issue)
  - E.g. the IP socket layer defines a default endianity (big endian) and it is up to the sender/received properly swap bytes

- Another issue is related to the usage of pointers in data structures because
  - This affects also communication on the same machine because different processes have a different address space (virtual addresses)
  - Pointers can be safely exchanged when communication among threads of the same process occurs

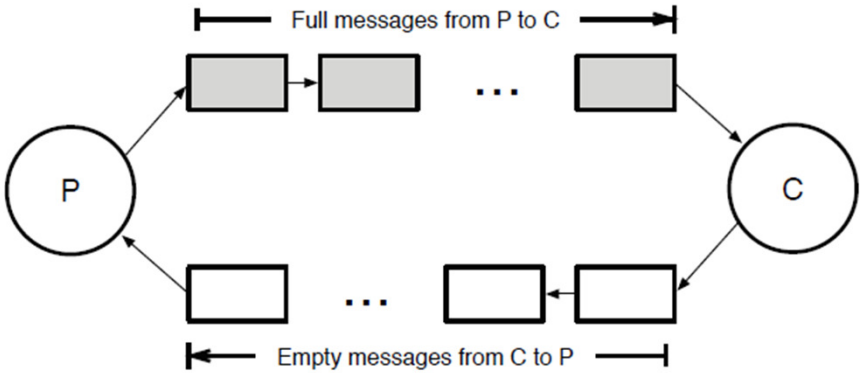
- A common practice when sending complex data structures is to serialize them in a sequence of bytes
  - If the structure contains internal pointers (e.g. a linked queue) the pointers are normally replaced by an offset from the first byte of the buffer in serialization and replaced by pointers in deserialization

Producer Consumer implemented with messages

```
#define N 8
void cons_init(void) {
  int i;
  empty_t s = empty;
  for(i=0; i<N; i++)
    send(P, &s, sizeof(s));
}
```

```
void prod(int d) {
  uint32_t m;
  empty_t s;
  recv(C, &s, sizeof(s));
  m = host_to_neutral(d);
  send(C, &m, sizeof(m));
}
```

```
int cons(void) {
  uint32_t m;
  empty_t s = empty;
  int d;
  recv(P, &m, sizeof(m));
  d = neutral_to_host(m);
  send(P, &s, sizeof(s));
  return d;
}
```



The programming model of TCP/IP

- In the programming model of TCP/IP, a socket must be established between the client and and the server.

- Once the socket has been established, client and servers can communicate, i.e. send and receive messages, that are ensured to be correctly delivered
  - Both client and server must adhere to a given protocol for meaningful communication
    - e.g if both issue e receive() call, they stop forever

- In order to establish a connection (socket), the client must specify the Internet Protocol (IP) address and the port number of the target server.

- The server instead is not aware of potential client until a connection has been established. The server therefore issues a accept() command that shall return when a client requested the establishment of a communication. At this point both client and server will receive a socket to be used for further read() and write() operations

- In order to be able to serve more than one client concurrently, the server shall start a new thread/process every time the accept() call returns, passing the returned socket to the task

The programming model of UDP

- Unlike TCP/IP, the programming model is message oriented, i.e. a UDP datagram (normally limited to 64Kbytes) can be sent and received
- Still in this case a socket must be created and used for sending and receiving datagrams, however in this case the successful socket creation does not imply the establishment of a connection (in this case the socket is in 'unconnected' state)
- Messages are sent via sendto() routine whose arguments shall specify the IP address and port number of the received.
- The a task wants to receive a datagram, it will call recv() or recvfrom(), after binding the socket to a port number
  - recv() shall return when a message has been received by any sender addressing this IP and port.
  - recvfrom() shall return when a message from the specified sender has been received
- UDP allows also the implementation of publish-subscribe pattern using UDP multicast.
  - In this case it is possible to register to a UDP multicast address, so that every message sent to this multicast address shall be received by all registered listeners